Modèles Algorithmiques Fondamentaux & Mécanismes d'Accès aux Collections

Thème principal : Initiation au développement (C++) - Analyse des structures de données et patrons algorithmiques canoniques

Synthèse exécutive : Ce cours pose les bases méthodologiques indispensables de l'algorithmique en BUT Informatique. Avant toute rédaction de code C++, le développeur doit impérativement analyser le contrat d'interface et le mécanisme d'accès (Direct, Séquentiel ou Dynamique) de la collection manipulée. Trois modèles algorithmiques canoniques sont formalisés pour les structures à accès direct : le parcours complet systématique, le parcours complet conditionné, et la recherche de première occurrence. Une rigueur absolue est exigée sur la gestion des bornes (indices de 0 à N-1 pour éviter les erreurs 'Out of Bounds'), le respect du principe du pessimisme algorithmique, ainsi que la présence systématique d'une double condition d'arrêt dans toute recherche.

Typologie et Mécanismes d'Accès aux Collections de Données

L'étape zéro de la conception algorithmique consiste à identifier le mode d'accès offert par la structure de données. Sur un tableau en mémoire vive (RAM) ou une chaîne de caractères, l'accès direct permet de pointer une case mémoire spécifique en calculant immédiatement son adresse offset à partir de l'indice. En revanche, sur un flux séquentiel (tel qu'un fichier texte ou CSV lu via un buffer disque), le système d'exploitation ne gère qu'un curseur de lecture glissant. Il est physiquement impossible de sauter directement à la 50ème ligne sans avoir lu les 49 précédentes. Tenter d'utiliser une notation indexée comme T[i] sur un flux séquentiel représente une erreur d'abstraction fondamentale.

- Structure de données : Collection organisée d'éléments de même type manipulable via des primitives dédiées.
- Accès direct : Capacité d'accéder instantanément à un élément via son indice T[i] en temps constant O(1).
- Accès séquentiel : Contrainte d'exploration pas à pas (début -> suivant) sans saut direct possible.
- Incompatibilité structurelle : Interdiction formelle d'appliquer une syntaxe d'indexation direct T[i] sur un flux séquentiel.

Cas concret enseignant : *Le professeur insiste sur l'erreur classique commise en contrôle : écrire T[i] sur un fichier CSV ouvert. Comme l'OS ne fournit qu'un curseur séquentiel, un tel code est sémantiquement faux et entraîne une note de zéro à la question.*

Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle est mobilisé lorsqu'aucun filtre ni aucune exclusion ne s'applique. Il sert par exemple à afficher tous les éléments d'un tableau, initialiser une structure ou modifier globalement les données. Le nombre d'éléments N étant fixé et connu avant l'entrée dans la boucle, le choix de la structure de contrôle se porte obligatoirement sur la boucle bornée 'Pour'. En langage C++ ainsi qu'en algorithmique standard, les tableaux étant indexés de 0 à N-1, le compteur d'itération 'i' doit démarrer à 0 et s'arrêter à N-1 inclusivement. Toute itération jusqu'à N constitue un dépassement de capacité mémoire (Out of Bounds) qui corrompt la mémoire ou fait crasher le programme.

- Traitement inconditionnel : Application uniforme d'une instruction à 100% des éléments.
- Structure itérative bornée : Utilisation de la boucle 'Pour' (nombre d'itérations connu à l'avance).
- Bornes d'indexation : Pour N éléments indexés à partir de 0, la boucle varie strictement de 0 à N-1.
- Erreur Out of Bounds : Tentative d'accès à l'indice N provoquant une violation d'accès mémoire.

Cas concret enseignant : *Lors de l'exemple 'Afficher un tableau de N cases' présenté en amphi, l'enseignant rappelle que la faute la plus récurrente des étudiants débutants est d'écrire 'Pour i allant de 0 à N', provoquant une erreur fatale à l'exécution lors de l'accès à la case T[N] inexistante.*

Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Ce modèle associe une exploration complète de la structure à un traitement sélectif. Chaque élément est visité via une boucle bornée 'Pour', mais l'instruction de traitement est placée à l'intérieur d'un bloc conditionnel 'Si (Condition) Alors'. Il est utilisé pour compter des occurrences, filtrer des affichages ou calculer des sommes partielles. Une vigilance accrue est requise lorsque le filtre implique des comparaisons entre voisins (ex: comparer T[i] et T[i-1]). Dans ce cas, pour éviter un débordement à l'indice -1 lors du premier tour, l'algorithme doit réajuster sa borne de départ à 1 au lieu de 0.

- Parcours exhaustif : Tous les éléments de la collection sont visités sans exception.
- Filtre logique (Prédicat) : L'action de traitement est subordonnée à l'évaluation d'un test 'Si ... Alors'.
- Gestion des éléments adjacents : Ajustement des bornes de boucle si la condition compare T[i] avec T[i-1] ou T[i+1].

Cas concret enseignant : *Analogie de la chaîne de montage en usine : chaque pièce défile obligatoirement devant le contrôleur qualité (parcours complet systématique), mais le contrôleur n'appose le tampon de rebut (action de traitement) que si et seulement si la pièce présente une fissure (traitement conditionné).*

Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

À la différence des parcours complets, un algorithme de recherche doit interrompre son exécution dès que la cible est localisée. On applique le principe du pessimisme algorithmique en initialisant un booléen 'trouvé' à Faux. La boucle 'Tant que' doit vérifier simultanément deux conditions logiques : ne pas dépasser la taille de la collection AND ne pas avoir encore trouvé l'élément ('i < N et NON trouvé'). Deux variantes existent : la Variante 1 sort immédiatement dès la découverte (conservant l'indice 'i' pile sur l'élément), tandis que la Variante 2 regroupe le test en tête de boucle mais peut décaler l'indice d'un pas si l'incrémentation est automatique.

- Pessimisme algorithmique : Postuler l'absence de l'élément avant le début du parcours ('trouvé = Faux').
- Boucle non bornée : Utilisation d'une boucle 'Tant que' (nombre de tours imprévisible).
- Double condition d'arrêt : Arrêt impératif dès que l'élément est trouvé OU que la collection est épuisée.
- Variante 1 vs Variante 2 : Différence d'implémentation entre rupture immédiate (maintien de l'indice exact) et Tant que global avec test en tête.

Cas concret enseignant : *Devise de l'enseignant : 'En informatique, soyez pessimistes ! Partez du principe que ce que vous cherchez n'existe pas, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il prévient que tout algorithme de recherche présenté avec une seule condition d'arrêt en examen est sanctionné par une note éliminatoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systematic sur une structure à accès dynamique ?
Réponse du professeur : Oui, tout à fait. Une structure à accès dynamique met à disposition des primitives permettant à la fois l'itération séquentielle et, selon la structure, l'accès indexé. On peut donc y dérouler un parcours complet, soit via un itérateur, soit à l'aide d'indices.

Question étudiant : Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : La Variante 1 (avec interruption directe) permet de conserver l'indice 'i' exact où le composant a été trouvé sans incrémentation parasite. La Variante 2 (Tant que global avec condition unique) respecte une structure à entrée/sortie unique conforme aux normes strictes de la programmation structurée, mais exige de prêter attention à la position de l'incrément.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut impérativement réajuster les bornes de la boucle pour ne jamais sortir de la mémoire allouée au tableau. Si le test compare T[i] à T[i-1], la boucle doit démarrer à i = 1 (jusqu'à N-1). Si le test compare T[i] à T[i+1], la boucle doit démarrer à i = 0 et s'arrêter à N-2.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur (par exemple trouver toutes les occurrences du nombre 5 dans un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il faut utiliser le Modèle 2 (Parcours complet avec traitement conditionnel) et non une recherche de première occurrence. En effet, l'objectif est d'explorer la collection jusqu'au bout pour répertorier ou afficher toutes les occurrences sans s'arrêter à la première.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictly positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Choix du modèle algorithmique. L'objectif exige d'examiner l'intégralité des cases du tableau T (structure à accès direct) de l'indice de départ à l'indice de fin. L'opération d'affichage est soumise à la condition 'valeur > 0'. Le modèle adéquat est donc le 'Parcours complet avec traitement conditionnel'. Étape 2 : Définition des bornes d'indexation. Pour un tableau de N éléments indexé à partir de 0, le premier élément est T[0] et le dernier est T[N-1]. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Structure d'itération et filtre. Le nombre de répétitions est connu : boucle bornée 'Pour i allant de 0 à N - 1'. À chaque itération, insertion du test conditionnel 'Si T[i] > 0 Alors'. Étape 4 : Rédaction de l'algorithme.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : Cet algorithme garantit un parcours exhaustif sans risque de débordement mémoire (arrêt strict à N-1) et filtre sélectivement les valeurs strictement positives.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on employer pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La Recherche de Première Occurrence (Modèle 3), car il faut interrompre le parcours dès que la valeur 10 est localisée au lieu de continuer à parcourir inutilement le reste du tableau.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il obligatoirement N - 1 et non N ?

R : Car l'indexation débute à 0. Les N éléments occupent ainsi les indices 0, 1, ..., N-1. Tenter d'accéder à T[N] provoque un dépassement de capacité mémoire ('out of bounds').

Q : Combien de conditions d'arrêt minimales doit comporter la structure de contrôle d'une recherche de première occurrence ?

R : Elle doit obligatoirement comporter 2 conditions d'arrêt : le succès de la recherche (élément trouvé) ET l'épuisement de la structure de données (fin du tableau atteinte).